Southern New Hampshire University

CS 330 Applying Lighting to a 3D Scene

Lighting is one of the most complex yet rewarding phases of photorealistic rendering. Many complex strategies can be used when performing lighting operations that will greatly increase the level of realism in scenes. In this course, the lighting phase is the final phase of scene enhancements.

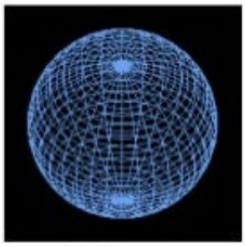
There are many levels of lighting techniques. The more technical the strategy, the higher the level of advanced functionality and the more realistic your effects. The lighting technique that is covered in this course is called the Phong lighting technique. With this technique, you can add three types of lights to scenes to provide realistic lighting effects. The three types of lights are ambient, diffuse, and specular.

Ambient lighting is the residual lighting that naturally occurs in the environment. Ambient lighting has no direction, reflectivity, or shadows. Ambient lighting is merely a certain level of light that falls evenly on all the objects in the 3D scene.

Diffuse lighting is soft, with no sharp contrast changes. An example of diffuse lighting is a floodlight or light source shining through a filter that softens the light.

Specular lighting is direct, unfiltered lighting that can generate bright reflectivity highlights from the objects it touches. The reflectivity level depends on the material of the object. An example of specular lighting is a theatrical spotlight that highlights different objects in a scene.

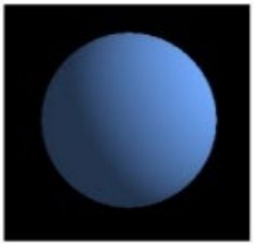
All three types of lighting are required to bring the highest level of realism to a 3D scene. The following picture shows the different levels achieved from the three types of lighting.



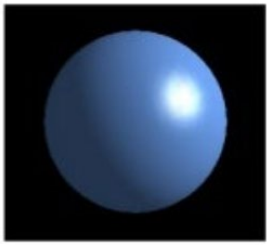
Wireframe

1. Original Color

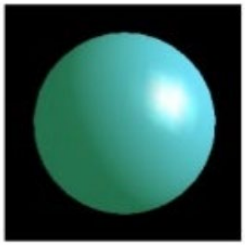
2. Original Color + Ambient



3. Original Color + Ambient + Diffuse

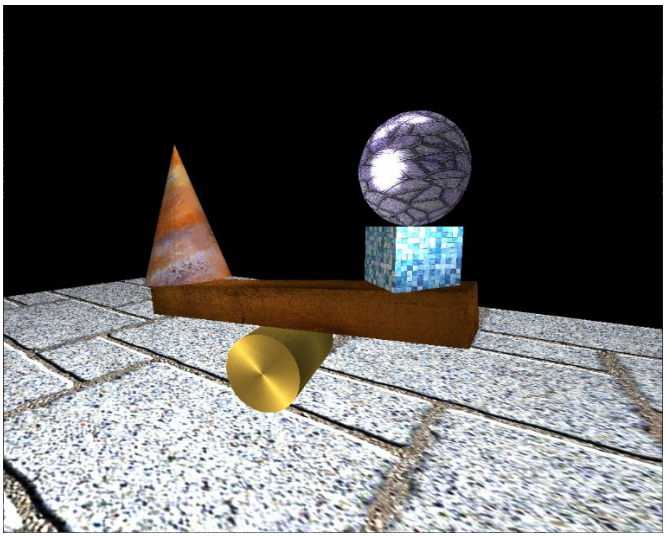


4. Original Color + Ambient + Diffuse + Specular



5. Original Color + Ambient + Diffuse + Specular + Emissive

The following picture shows a 3D scene that has been enhanced by lighting. The picture also includes texture mapping on the various objects.



Each object in the scene above is identified by different types of materials to add more realism to the light interactions.

- The ball is a glass material with much shininess and specular reflection.
- The box under the ball is made of a tile material. The tile material doesn't have as much reflectivity, but it has glossy highlights.
- The plank balanced on the cylinder is a wood material with no specular reflectivity. Soft, diffuse lighting brings out the color of the wood.
- The cone is a clay object with no shininess but with soft and colorful lighting.
- The bottom cylinder is a gold metal object with patterned specular reflectivity and diffuse shading that highlights the rich color.
- The bottom plane is a cement material with some brightness and a mostly rough appearance.

To implement the Phong lighting technique with the provided CS330Content source code, most of the code changes are included in the SceneManager C++ class. You must use several new methods to achieve the various materials and lighting setup and configuration tasks required for the 3D scene.

```
/***********************************************************
 *  PrepareScene()
 *
 *  This method is used for preparing the 3D scene by loading
 *  the shapes, textures in memory to support the 3D scene
 *  rendering
 ***********************************************************/
void SceneManager::PrepareScene()
{
	LoadSceneTextures();
	DefineObjectMaterials();
```

```
        SetupSceneLights();

        // only one instance of a particular mesh needs to be
        // loaded in memory no matter how many times it is drawn
        // in the rendered 3D scene

        m_basicMeshes->LoadBoxMesh();
        m_basicMeshes->LoadPlaneMesh();
        m_basicMeshes->LoadCylinderMesh();
        m_basicMeshes->LoadConeMesh();
        m_basicMeshes->LoadSphereMesh();
```

The two new methods that need to be called from the PrepareScene() method are DefineObjectMaterials() and SetupSceneLights(). Both methods need to be called once before the scene rendering starts. Look closer at each of the new methods below.

```
/***********************************************************
 *  DefineObjectMaterials()
 *
 *  This method is used for configuring the various material
 *  settings for all of the objects in the 3D scene.
 ***********************************************************/
void SceneManager::DefineObjectMaterials()
{
        OBJECT_MATERIAL goldMaterial;
        goldMaterial.ambientColor = glm::vec3(0.2f, 0.2f, 0.1f);
        goldMaterial.ambientStrength = 0.4f;
        goldMaterial.diffuseColor = glm::vec3(0.3f, 0.3f, 0.2f);
        goldMaterial.specularColor = glm::vec3(0.6f, 0.5f, 0.4f);
        goldMaterial.shininess = 22.0;
        goldMaterial.tag = "gold";

        m_objectMaterials.push_back(goldMaterial);

        OBJECT_MATERIAL cementMaterial;
        cementMaterial.ambientColor = glm::vec3(0.2f, 0.2f, 0.2f);
        cementMaterial.ambientStrength = 0.2f;
        cementMaterial.diffuseColor = glm::vec3(0.5f, 0.5f, 0.5f);
        cementMaterial.specularColor = glm::vec3(0.4f, 0.4f, 0.4f);
        cementMaterial.shininess = 0.5;
        cementMaterial.tag = "cement";

        m_objectMaterials.push_back(cementMaterial);

        OBJECT_MATERIAL woodMaterial;
        woodMaterial.ambientColor = glm::vec3(0.4f, 0.3f, 0.1f);
        woodMaterial.ambientStrength = 0.2f;
```

```
	woodMaterial.diffuseColor = glm::vec3(0.3f, 0.2f, 0.1f);
	woodMaterial.specularColor = glm::vec3(0.1f, 0.1f, 0.1f);
	woodMaterial.shininess = 0.3;
	woodMaterial.tag = "wood";

	m_objectMaterials.push_back(woodMaterial);

	OBJECT_MATERIAL tileMaterial;
	tileMaterial.ambientColor = glm::vec3(0.2f, 0.3f, 0.4f);
	tileMaterial.ambientStrength = 0.3f;
	tileMaterial.diffuseColor = glm::vec3(0.3f, 0.2f, 0.1f);
	tileMaterial.specularColor = glm::vec3(0.4f, 0.5f, 0.6f);
	tileMaterial.shininess = 25.0;
	tileMaterial.tag = "tile";

	m_objectMaterials.push_back(tileMaterial);

	OBJECT_MATERIAL glassMaterial;
	glassMaterial.ambientColor = glm::vec3(0.4f, 0.4f, 0.4f);
	glassMaterial.ambientStrength = 0.3f;
	glassMaterial.diffuseColor = glm::vec3(0.3f, 0.3f, 0.3f);
	glassMaterial.specularColor = glm::vec3(0.6f, 0.6f, 0.6f);
	glassMaterial.shininess = 85.0;
	glassMaterial.tag = "glass";

	m_objectMaterials.push_back(glassMaterial);

	OBJECT_MATERIAL clayMaterial;
	clayMaterial.ambientColor = glm::vec3(0.2f, 0.2f, 0.3f);
	clayMaterial.ambientStrength = 0.3f;
	clayMaterial.diffuseColor = glm::vec3(0.4f, 0.4f, 0.5f);
	clayMaterial.specularColor = glm::vec3(0.2f, 0.2f, 0.4f);
	clayMaterial.shininess = 0.5;
	clayMaterial.tag = "clay";

	m_objectMaterials.push_back(clayMaterial);
}
```

The DefineObjectMaterials() is used to identify the properties of the materials for the objects in the 3D scene. Multiple objects in the scene may be made from the same material. The type of material assigned to each object will determine how the various light sources in the scene interact with the object. You want the objects to look as real as possible. You also want the objects to look like the material they are made from.

```
void SceneManager::SetupSceneLights()
{
	m_pShaderManager->setVec3Value("lightSources[0].position", 3.0f, 14.0f, 0.0f);
```

```
    m_pShaderManager->setVec3Value("lightSources[0].ambientColor", 0.01f, 0.01f, 0.01f);
    m_pShaderManager->setVec3Value("lightSources[0].diffuseColor", 0.4f, 0.4f, 0.4f);
    m_pShaderManager->setVec3Value("lightSources[0].specularColor", 0.0f, 0.0f, 0.0f);
    m_pShaderManager->setFloatValue("lightSources[0].focalStrength", 32.0f);
    m_pShaderManager->setFloatValue("lightSources[0].specularIntensity", 0.05f);

    m_pShaderManager->setVec3Value("lightSources[1].position", -3.0f, 14.0f, 0.0f);
    m_pShaderManager->setVec3Value("lightSources[1].ambientColor", 0.01f, 0.01f, 0.01f);
    m_pShaderManager->setVec3Value("lightSources[1].diffuseColor", 0.4f, 0.4f, 0.4f);
    m_pShaderManager->setVec3Value("lightSources[1].specularColor", 0.0f, 0.0f, 0.0f);
    m_pShaderManager->setFloatValue("lightSources[1].focalStrength", 32.0f);
    m_pShaderManager->setFloatValue("lightSources[1].specularIntensity", 0.05f);

    m_pShaderManager->setVec3Value("lightSources[2].position", 0.6f, 5.0f, 6.0f);
    m_pShaderManager->setVec3Value("lightSources[2].ambientColor", 0.01f, 0.01f, 0.01f);
    m_pShaderManager->setVec3Value("lightSources[2].diffuseColor", 0.3f, 0.3f, 0.3f);
    m_pShaderManager->setVec3Value("lightSources[2].specularColor", 0.3f, 0.3f, 0.3f);
    m_pShaderManager->setFloatValue("lightSources[2].focalStrength", 12.0f);
    m_pShaderManager->setFloatValue("lightSources[2].specularIntensity", 0.5f);

    m_pShaderManager->setBoolValue("bUseLighting", true);
}
```

The SetupSceneLights() method is used to add and configure the various light sources that will add more realism to the 3D scene. There are a total of four light sources in the fragment shader code that can be defined and added to a 3D scene. Three light sources have been added to the source code above. For the fragment shader to render the scene with the added light sources, the bUseLighting variable must be set to true, as seen in the last line of code.

Six properties can be set for each material to define the attributes of the material. These properties are as follows:

- **ambientColor** is a vec3 color value that can be used to set the ambient emission color.
- **ambientStrength** is a float value that defines how much ambient light is emitted.
- **diffuseColor** is a vec3 color value that defines the diffuse light emission color.
- **specularColor** is a vec3 color value that defines the specular light reflection color.
- **shininess** is a float value that tells how shiny the material is and how much specular light will be reflected off the surface.
- **tag** is a special string that is used to identify the material in the collection.

Six properties that can be used for each light source to define the attributes of the light source. These properties are as follows:

- **position** is a vec3 value that sets the position of the light source in 3D space.
- **ambientColor** is a vec3 value that defines the color of the ambient light being emitted.
- **diffuseColor** is a vec3 value that defines the color of the diffuse light being emitted.

- **specularColor** is a vec3 value that defines the color of the specular light being emitted.
- **focalStrength** is a float value that defines the focus of the light beam being emitted.
- **specularIntensity** is a float value that defines the intensity of the emitted specular lighting.

The way the new code is being used in the RenderScene() method is shown below.

```
/***********************************************************
 *  RenderScene()
 *
 *  This method is used for rendering the 3D scene by
 *  transforming and drawing the basic 3D shapes
 ***********************************************************/
void SceneManager::RenderScene()
{
	// declare the variables for the transformations
	glm::vec3 scaleXYZ;
	float XrotationDegrees = 0.0f;
	float YrotationDegrees = 0.0f;
	float ZrotationDegrees = 0.0f;
	glm::vec3 positionXYZ;

	glm::mat4 scale;
	glm::mat4 rotation;
	glm::mat4 rotation2;
	glm::mat4 translation;
	glm::mat4 model;

	/*** Set needed transformations before drawing the basic mesh ***/

	// set the XYZ scale for the mesh
	scaleXYZ = glm::vec3(20.0f, 1.0f, 10.0f);

	// set the XYZ rotation for the mesh
	XrotationDegrees = 0.0f;
	YrotationDegrees = 0.0f;
	ZrotationDegrees = 0.0f;

	// set the XYZ position for the mesh
	positionXYZ = glm::vec3(0.0f, 0.0f, 0.0f);

	// set the transformations into memory to be used on the drawn meshes
	SetTransformations(
		scaleXYZ,
		XrotationDegrees,
		YrotationDegrees,
		ZrotationDegrees,
		positionXYZ);
```

```
//SetShaderColor(1, 1, 1, 1);
SetShaderTexture("floor");
SetShaderMaterial("cement");

// draw the mesh with transformation values - this plane is used for the base
m_basicMeshes->DrawPlaneMesh();

/*** Set needed transformations before drawing the basic mesh ***/

scaleXYZ = glm::vec3(0.9f, 2.5f, 0.9f);

// set the XYZ rotation for the mesh
XrotationDegrees = 90.0f;
YrotationDegrees = 0.0f;
ZrotationDegrees = -15.0f;

// set the XYZ position for the mesh
positionXYZ = glm::vec3(0.0f, 0.9f, 0.0f);

// set the transformations into memory to be used on the drawn meshes
SetTransformations(
        scaleXYZ,
        XrotationDegrees,
        YrotationDegrees,
        ZrotationDegrees,
        positionXYZ);

//SetShaderColor(1, 0, 0, 1);
SetShaderTexture("cylinder");
SetShaderMaterial("gold");

m_basicMeshes->DrawCylinderMesh(false, false);

SetShaderTexture("cylinder_top");

m_basicMeshes->DrawCylinderMesh(true, true, false);

scaleXYZ = glm::vec3(0.9f, 9.0f, 1.2f);

// set the XYZ rotation for the mesh
XrotationDegrees = 0.0f;
YrotationDegrees = 0.0f;
ZrotationDegrees = 95.0f;

// set the XYZ position for the mesh
positionXYZ = glm::vec3(0.2f, 2.25f, 1.4f);
```

```
// set the transformations into memory to be used on the drawn meshes
SetTransformations(
        scaleXYZ,
        XrotationDegrees,
        YrotationDegrees,
        ZrotationDegrees,
        positionXYZ);

//SetShaderColor(0, 0, 1, 1);
SetShaderTexture("plank");
SetShaderMaterial("wood");

m_basicMeshes->DrawBoxMesh();

scaleXYZ = glm::vec3(1.3f, 1.1f, 1.3f);

// set the XYZ rotation for the mesh
XrotationDegrees = 0.0f;
YrotationDegrees = 48.0f;
ZrotationDegrees = 0.0f;

// set the XYZ position for the mesh
positionXYZ = glm::vec3(3.5f, 3.55f, 1.3f);

// set the transformations into memory to be used on the drawn meshes
SetTransformations(
        scaleXYZ,
        XrotationDegrees,
        YrotationDegrees,
        ZrotationDegrees,
        positionXYZ);

//SetShaderColor(1, 0, 1, 1);
SetShaderTexture("box");
SetShaderMaterial("tile");

m_basicMeshes->DrawBoxMesh();

scaleXYZ = glm::vec3(1.0f, 1.0f, 1.0f);

// set the XYZ rotation for the mesh
XrotationDegrees = 0.0f;
YrotationDegrees = 0.0f;
ZrotationDegrees = 0.0f;

// set the XYZ position for the mesh
```

```
	positionXYZ = glm::vec3(3.5f, 5.10f, 1.3f);

	// set the transformations into memory to be used on the drawn meshes
	SetTransformations(
		scaleXYZ,
		XrotationDegrees,
		YrotationDegrees,
		ZrotationDegrees,
		positionXYZ);

	//SetShaderColor(1, 1, 0, 1);
	SetShaderTexture("ball");
	SetShaderMaterial("glass");

	m_basicMeshes->DrawSphereMesh();

	scaleXYZ = glm::vec3(1.2f, 4.0f, 1.2f);

	// set the XYZ rotation for the mesh
	XrotationDegrees = 0.0f;
	YrotationDegrees = 0.0f;
	ZrotationDegrees = 5.0f;

	// set the XYZ position for the mesh
	positionXYZ = glm::vec3(-3.2f, 2.48f, 1.4f);

	// set the transformations into memory to be used on the drawn meshes
	SetTransformations(
		scaleXYZ,
		XrotationDegrees,
		YrotationDegrees,
		ZrotationDegrees,
		positionXYZ);

	//SetShaderColor(0, 1, 0, 1);
	SetShaderTexture("cone");
	SetShaderMaterial("clay");

	m_basicMeshes->DrawConeMesh();
}
```